

# SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC HOMEWORK & ASSESSMENT SUBMISSION

|                                 |                                                                                          |
|---------------------------------|------------------------------------------------------------------------------------------|
| <b>Student Name:</b>            | RAM CHARAN VANGA                                                                         |
| <b>Register No:</b>             | RA2511027010164                                                                          |
| <b>Course Code &amp; Title:</b> | STEP-JAVA - SRM STUDENT TALENT ENHANCEMENT PROGRAM (JAVA TRACK)                          |
| <b>Task &amp; Channel:</b>      | STEP Java Challenge: Polymorphic Banking Transaction & Custom Exceptions [SRM_STEP_JAVA] |

## Assignment Description:

Design a robust BankAccount hierarchy with SavingsAccount and CurrentAccount. Implement deposit, withdraw, OverdraftLimitExceededException, and InsufficientBalanceException. Use Java Generics and Streams to filter accounts with balance > 50,000.

## Technical Solution:

### Solution Report: SRM STEP Java Challenge

\*\*Student:\*\* RAM CHARAN VANGA (RA2511027010164)

\*\*Module:\*\* Object-Oriented Programming, Custom Exception Handling, and Java 8 Streams API

#### 1. Architecture Overview:

- `BankAccount` provides an abstract base with protected member variables ensuring proper encapsulation.
- `SavingsAccount` implements strict minimum balance enforcement (`MIN\_BALANCE = 1000.0`) and calculates compound interest.
- `CurrentAccount` supports flexible working capital with custom overdraft limit verification.
- Domain exceptions `InsufficientBalanceException` and `OverdraftLimitExceededException` enforce transaction integrity.
- `BankManager` demonstrates functional programming using Java 8 Stream filters, comparators, and parallel reductions.

#### 2. Test Execution Results:

- \*\*Test Case 1 (Valid Withdrawal):\*\* PASSED (Withdrawal within bounds).
- \*\*Test Case 2 (Minimum Balance Violation):\*\* PASSED (`InsufficientBalanceException` triggered).
- \*\*Test Case 3 (Overdraft Boundary):\*\* PASSED (`OverdraftLimitExceededException` triggered).
- \*\*Test Case 4 (Streams High-Value Filter):\*\* PASSED (Filtered...

## Verified Source Code Deliverables:

**File: BankAccount.java**

```
// =====  
// SRM STEP Java Track: BankAccount.java  
// Package: com.srm.step.banking  
// Student: RAM CHARAN VANGA (RA2511027010164)  
// =====  
package com.srm.step.banking;
```

```

public abstract class BankAccount {
    protected String accountNumber;
    protected String accountHolder;
    protected double balance;

    public BankAccount(String accountNumber, String accountHolder, double balance) {
        if (balance < 0) {
            throw new IllegalArgumentException("Initial balance cannot be negative.");
        }
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    public String getAccountNumber() { return accountNumber; }
    public String
// ... [Full code attached in repository]

```

### File: SavingsAccount.java

```

// =====
// SRM STEP Java Track: SavingsAccount.java
// =====
package com.srm.step.banking;

public class SavingsAccount extends BankAccount {
    private static final double MIN_BALANCE = 1000.0;
    private double interestRate;

    public SavingsAccount(String accountNumber, String accountHolder, double balance, double interestRate) {
        super(accountNumber, accountHolder, balance);
        this.interestRate = interestRate;
    }

    @Override
    public void withdraw(double amount) throws InsufficientBalanceException {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive.");
        }
        if (balance - amount < MIN_BALANCE) {
            throw new
// ... [Full code attached in repository]

```